

Nhân đa thức nhanh

Zhang Jialin

Nguồn gốc: Fast polynomial multiplication

IOI China candidate team papers / 2002 / Zhang Jialin.pdf

Contents

1	Dẫn nhập	1
2	Các phép toán cơ bản trên đa thức	2
3	Phép nhân đa thức thông thường	2
4	Biểu diễn điểm–giá trị của đa thức	2
5	Nhân đa thức trong biểu diễn điểm–giá trị	3
6	Cải thiện phép nhân hệ số bằng biểu diễn điểm–giá trị	4
7	Từ hệ số sang điểm–giá trị	4
8	Từ điểm–giá trị về hệ số	5
9	Luồng thuật toán nhân đa thức	7
10	Cải tiến thuật toán	7
11	So sánh thực nghiệm	8

1 Dẫn nhập

Đa thức là một trong những công cụ toán học cơ bản nhất. Vì có dạng đơn giản và dễ thực hiện các phép tính trên máy tính, đa thức ngày càng được dùng rộng rãi. Nó không chỉ giữ vai trò quan trọng trong các phần mềm toán học như Maple, mà còn có nhiều ứng dụng trong kỹ thuật, thông tin và nhiều lĩnh vực khác.

Sau đây là một vài ví dụ về xấp xỉ bằng đa thức:

$$\ln(x+1) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots + (-1)^{n-1} \frac{x^n}{n} + \cdots,$$
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots + (-1)^n \frac{x^{2n+1}}{(2n+1)!} + \cdots,$$

Bổ đề 1: tính duy nhất của nội suy đa thức. Với tập bất kỳ gồm n cặp điểm–giá trị

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\},$$

trong đó $x_0, x_1, \dots, x_{n-1}$ đôi một khác nhau, tồn tại duy nhất một đa thức $A(x)$ bậc không quá $n - 1$ sao cho

$$y_k = A(x_k), \quad k = 0, 1, \dots, n - 1.$$

Chứng minh tồn tại. Đặt

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}.$$

Điều kiện $A(x_k) = y_k$ tương đương hệ phương trình tuyến tính

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}.$$

Ký hiệu hệ này là $XA = Y$. Ma trận X là ma trận Vandermonde. Bằng các phép biến đổi định thức, ta có

$$\det X = \prod_{j < k} (x_k - x_j) \neq 0.$$

Vì vậy X khả nghịch và $A = X^{-1}Y$ tồn tại.

Chứng minh duy nhất. Giả sử hai đa thức $f(x)$ và $g(x)$, đều có bậc không quá $n - 1$, cùng thỏa các cặp điểm–giá trị trên. Khi đó

$$r(x) = f(x) - g(x)$$

có n nghiệm $x_0, x_1, \dots, x_{n-1}$, trong khi $r(x)$ có bậc không quá $n - 1$. Suy ra $r(x) \equiv 0$, tức $f(x) = g(x)$.

5 Nhân đa thức trong biểu diễn điểm–giá trị

Nếu $r(x) = f(x)g(x)$, thì tại từng điểm x_i ,

$$r(x_i) = f(x_i)g(x_i).$$

Nói cách khác, nếu ta có ba biểu diễn điểm–giá trị dùng cùng các hoành độ,

$$(x_i, r_i), \quad (x_i, f_i), \quad (x_i, g_i),$$

thì

$$r_i = f_i g_i.$$

Vì vậy, nếu dùng đúng cách biểu diễn điểm–giá trị, phép nhân đa thức chỉ cần thời gian tuyến tính.

Tuy nhiên, nếu f và g đều có bậc không quá $n - 1$, thì $r = fg$ có bậc không quá $2n - 2$. Để xác định duy nhất $r(x)$, ta cần $2n - 1$ cặp điểm–giá trị, trong khi ban đầu chỉ có n cặp. Ta giải quyết bằng cách mở rộng số cặp điểm–giá trị của $f(x)$ và $g(x)$ ngay từ đầu lên $2n - 1$.

6 Cải thiện phép nhân hệ số bằng biểu diễn điểm–giá trị

Ta muốn tận dụng việc nhân trong biểu diễn điểm–giá trị chỉ cần thời gian tuyến tính để tăng tốc phép nhân đa thức vốn được cho bằng hệ số. Cần làm ba bước:

1. chuyển từ biểu diễn hệ số sang biểu diễn điểm–giá trị, gọi là quá trình tính giá trị;
2. nhân từng điểm trong biểu diễn điểm–giá trị;
3. chuyển từ biểu diễn điểm–giá trị về biểu diễn hệ số, gọi là nội suy.

Bước thứ hai chỉ cần thời gian tuyến tính. Mấu chốt là bước thứ nhất và bước thứ ba.

7 Từ hệ số sang điểm–giá trị

Ta cần tính

$$x_i \mapsto f(x_i), \quad i = 0, 1, \dots, n-1.$$

Nếu dùng Horner trực tiếp thì

$$f(x_i) = ((\dots (a_{n-1}x_i + a_{n-2})x_i + a_{n-3}) \dots + a_1)x_i + a_0.$$

Điểm quan trọng là các $x_0, x_1, \dots, x_{n-1}$ do ta tự chọn. Bằng cách chọn phù hợp, ta có thể đưa quá trình chuyển đổi xuống $O(n \log n)$.

Ta chọn các căn bậc n của 1:

$$\varepsilon_n^0, \varepsilon_n^1, \dots, \varepsilon_n^{n-1},$$

trong đó

$$\varepsilon_n^k = e^{2\pi i k/n} = \cos \frac{2\pi k}{n} + i \sin \frac{2\pi k}{n}.$$

Bổ đề 2: định lý chia đôi. Với mọi số nguyên $n \geq 0$ và $k \geq 0$, ta có

$$(\varepsilon_{2n}^{2k}) = \varepsilon_n^k.$$

Chứng minh.

$$\varepsilon_{2n}^{2k} = \cos \frac{2\pi \cdot 2k}{2n} + i \sin \frac{2\pi \cdot 2k}{2n} = \cos \frac{2\pi k}{n} + i \sin \frac{2\pi k}{n} = \varepsilon_n^k.$$

Bài toán cần giải là tính

$$f(\varepsilon_n^k) = a_0 + a_1 \varepsilon_n^k + \dots + a_{n-1} (\varepsilon_n^k)^{n-1}, \quad k = 0, 1, \dots, n-1.$$

Giả sử n chẵn; nếu không, ta thêm các hệ số bậc cao bằng 0 để đưa n về số chẵn. Đặt

$$f^{[0]}(x) = a_0 + a_2 x + \dots + a_{n-2} x^{n/2-1},$$

$$f^{[1]}(x) = a_1 + a_3 x + \dots + a_{n-1} x^{n/2-1}.$$

Đa thức $f^{[0]}$ chứa mọi hệ số chỉ số chẵn của f , còn $f^{[1]}$ chứa mọi hệ số chỉ số lẻ. Khi đó

$$f(x) = f^{[0]}(x^2) + x f^{[1]}(x^2).$$

Vì vậy, để tính f tại $\varepsilon_n^0, \varepsilon_n^1, \dots, \varepsilon_n^{n-1}$, ta cần tính $f^{[0]}$ và $f^{[1]}$ tại các điểm

$$(\varepsilon_n^0)^2, (\varepsilon_n^1)^2, \dots, (\varepsilon_n^{n-1})^2.$$

Theo định lý chia đôi,

$$(\varepsilon_n^0)^2, (\varepsilon_n^1)^2, \dots, (\varepsilon_n^{n-1})^2 = \varepsilon_{n/2}^0, \varepsilon_{n/2}^1, \dots, \varepsilon_{n/2}^{n-1} \\ = \varepsilon_{n/2}^0, \varepsilon_{n/2}^1, \dots, \varepsilon_{n/2}^{n/2-1}, \varepsilon_{n/2}^0, \dots, \varepsilon_{n/2}^{n/2-1}.$$

Nghĩa là các bình phương trên không phải n số khác nhau; chúng chỉ gồm các căn bậc $n/2$ của 1, mỗi căn xuất hiện đúng hai lần. Do đó bài toán con có cùng dạng với bài toán gốc nhưng kích thước giảm một nửa. Điều này dẫn đến cách giải chia để trị, chính là ý tưởng của FFT.

Thuật toán đệ quy. Trong đoạn giả mã sau, a là dãy hệ số, y là dãy giá trị tại các căn đơn vị tương ứng.

```
Function transform(a: atype): ytype;
  if n = 1 then
    return a[0]

  if odd(n) then
    begin
      inc(n);
      a[n-1] := 0;
    end;

  a[0] := (a[0], a[2], ..., a[n-2]);
  a[1] := (a[1], a[3], ..., a[n-1]);

  y[0] := transform(a[0]);
  y[1] := transform(a[1]);

  for k := 0 to n-1 do
    y[k] := y[0][k mod (n/2)]
      + eps_n^k * y[1][k mod (n/2)];

  return y;
```

Trong công thức ở vòng lặp,

$$y(\varepsilon_n^k) = y^{[0]}(\varepsilon_{n/2}^k) + \varepsilon_n^k y^{[1]}(\varepsilon_{n/2}^k),$$

với chỉ số ở vế phải hiểu theo chu kỳ $n/2$. Có thể chứng minh cách tính này có độ phức tạp $O(n \log n)$.

8 Từ điểm-giá trị về hệ số

Nội suy là phép ngược của quá trình tính giá trị. Bài toán này có vẻ phức tạp hơn, nhưng thực ra có thể chuyển về bài toán trước.

Quá trình tính giá trị có thể viết thành phương trình ma trận

$$Y = V_n A,$$

trong đó

$$Y = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix}, \quad A = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix},$$

và

$$V_n = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \varepsilon_n & \varepsilon_n^2 & \varepsilon_n^3 & \dots & \varepsilon_n^{n-1} \\ 1 & \varepsilon_n^2 & \varepsilon_n^4 & \varepsilon_n^6 & \dots & \varepsilon_n^{2(n-1)} \\ 1 & \varepsilon_n^3 & \varepsilon_n^6 & \varepsilon_n^9 & \dots & \varepsilon_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \varepsilon_n^{n-1} & \varepsilon_n^{2(n-1)} & \varepsilon_n^{3(n-1)} & \dots & \varepsilon_n^{(n-1)(n-1)} \end{bmatrix}.$$

Nếu V_n khả nghịch, nội suy là

$$A = V_n^{-1}Y.$$

Bổ đề 3. Với ma trận V_n ở trên,

$$V_n^{-1} = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \varepsilon_n^{-1} & \varepsilon_n^{-2} & \varepsilon_n^{-3} & \dots & \varepsilon_n^{-(n-1)} \\ 1 & \varepsilon_n^{-2} & \varepsilon_n^{-4} & \varepsilon_n^{-6} & \dots & \varepsilon_n^{-2(n-1)} \\ 1 & \varepsilon_n^{-3} & \varepsilon_n^{-6} & \varepsilon_n^{-9} & \dots & \varepsilon_n^{-3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \varepsilon_n^{-(n-1)} & \varepsilon_n^{-2(n-1)} & \varepsilon_n^{-3(n-1)} & \dots & \varepsilon_n^{-(n-1)(n-1)} \end{bmatrix}.$$

Từ bổ đề này, ta giải được nội suy bằng cùng một thủ tục như quá trình tính giá trị, chỉ thay các căn ε_n^k bằng ε_n^{-k} , rồi chia mọi hệ số thu được cho n :

$$A = \frac{1}{n} V_n(\varepsilon_n^{-1})Y.$$

Thuật toán nội suy.

```
Function inverse_transform(y: ytype): atype;
  if n = 1 then
    return y0

  if odd(n) then
    begin
      inc(n);
      y[n-1] := 0;
    end;

  y[0] := (y0, y2, ..., y[n-2]);
  y[1] := (y1, y3, ..., y[n-1]);

  a[0] := inverse_transform(y[0]);
  a[1] := inverse_transform(y[1]);
```

```

for k := 0 to n-1 do
  a[k] := a[0][k mod (n/2)]
           + eps_n(-k) * a[1][k mod (n/2)];

after the recursion, divide every a[k] by n;
return a;

```

9 Luồng thuật toán nhân đa thức

Bài toán: $f(x)$ và $g(x)$ là hai đa thức bậc không quá $n - 1$, đã biết biểu diễn hệ số của chúng. Hãy tính biểu diễn hệ số của

$$r(x) = f(x)g(x).$$

Luồng thuật toán:

1. **Tiền xử lý.** Thêm $n - 1$ hệ số bậc cao bằng 0, để bậc có thể đạt đến $2n - 2$. Mục đích là có đủ số cặp điểm–giá trị để xác định duy nhất $r(x)$.
2. **Tính giá trị.** Dùng chia để trị qua hàm transform để tính giá trị của $f(x)$ và $g(x)$ tại các căn bậc $2n - 1$ của 1. Trong cài đặt FFT thường đệ quy tiếp đến một lũy thừa của 2 không nhỏ hơn $2n - 1$.
3. **Nhân từng điểm.** Với mỗi điểm, nhân giá trị của f và g để thu được giá trị của r tại điểm đó.
4. **Nội suy.** Hoán đổi vai trò của a và y , dùng lại transform với căn nghịch đảo để tìm biểu diễn hệ số của $r(x)$, rồi chia kết quả cho số điểm.

Bước 1 và bước 3 đều chạy trong $O(n)$. Bước 2 và bước 4 đều chạy trong $O(n \log n)$.

10 Cải tiến thuật toán

Trong hàm transform, ta đang dùng đệ quy. Với $n = 8$, quá trình gọi đệ quy tách các hệ số như sau:

$$\begin{array}{ccccccc}
 & & (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7) & & & & \\
 & \swarrow & & \searrow & & & \\
 (a_0, a_2, a_4, a_6) & & & & (a_1, a_3, a_5, a_7) & & \\
 & \swarrow \quad \searrow & & \swarrow \quad \searrow & & & \\
 (a_0, a_4) & (a_2, a_6) & & (a_1, a_5) & (a_3, a_7) & & \\
 & \downarrow \quad \downarrow & & \downarrow \quad \downarrow & & & \\
 (a_0) & (a_4) & (a_2) & (a_6) & (a_1) & (a_5) & (a_3) & (a_7).
 \end{array}$$

Phương pháp đệ quy cần nhiều bộ nhớ, vì mỗi lần gọi lại cần truyền một mảng hệ số mới vào quá trình con. Từ sơ đồ trên, ta thấy có thể làm ngược lại: thực hiện từ dưới lên bằng phép lặp. Thứ tự lá là

$$(a_0), (a_4), (a_2), (a_6), (a_1), (a_5), (a_3), (a_7),$$

rồi lần lượt ghép thành

$$(a_0, a_4), (a_2, a_6), (a_1, a_5), (a_3, a_7),$$

tiếp đến

$$(a_0, a_2, a_4, a_6), \quad (a_1, a_3, a_5, a_7),$$

và cuối cùng trở lại

$$(a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7).$$

Thuật toán lặp.

```
Function transform_better(a: atype): ytype;
  preprocess:
    add high-degree zero coefficients until the length is a power of 2

  initialize:
    y := a

  iterate:
    for k := 1 to lg n do
      merge suitable adjacent blocks and put the result
      into the corresponding positions of y

  return y
```

Với $n = 8$, các tầng lặp có thể mô tả như sau:

$$\begin{array}{cccccc}
 (a_0) & (a_4) & (a_2) & (a_6) & (a_1) & (a_5) & (a_3) & (a_7) \\
 \varepsilon_1^0 & \varepsilon_1^0 & \varepsilon_1^0 & \varepsilon_1^0 & \varepsilon_1^0 & \varepsilon_1^0 & \varepsilon_1^0 & \varepsilon_1^0 \\
 \\
 (a_0, a_4) & & (a_2, a_6) & & (a_1, a_5) & & (a_3, a_7) & \\
 \varepsilon_2^0, \varepsilon_2^1 & & \varepsilon_2^0, \varepsilon_2^1 & & \varepsilon_2^0, \varepsilon_2^1 & & \varepsilon_2^0, \varepsilon_2^1 & \\
 \\
 (a_0, a_2, a_4, a_6) & & & & (a_1, a_3, a_5, a_7) & & & \\
 \varepsilon_4^0, \varepsilon_4^1, \varepsilon_4^2, \varepsilon_4^3 & & & & \varepsilon_4^0, \varepsilon_4^1, \varepsilon_4^2, \varepsilon_4^3 & & & \\
 \\
 (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7) & & & & & & & \\
 \varepsilon_8^0, \varepsilon_8^1, \varepsilon_8^2, \varepsilon_8^3, \varepsilon_8^4, \varepsilon_8^5, \varepsilon_8^6, \varepsilon_8^7. & & & & & & &
 \end{array}$$

11 So sánh thực nghiệm

Cuối cùng, ta so sánh phương pháp được giới thiệu trong bài với phép nhân đa thức thông thường.

Môi trường thử nghiệm: PIII 500, 128M RAM, FreePascal 1.0.4.

Bậc đa thức	$n = 100$	$n = 1000$	$n = 10000$	$n = 20000$	$n = 30000$
Thường (s)	0.02	0.07	5.02	18.06	> 30
Điểm-giá trị (s)	0.05	0.50	2.46	2.88	3.36
Chính xác real	10–11 chữ số	10–11 chữ số	9–10 chữ số	8–9 chữ số	8–9 chữ số

Bảng cho thấy khi bậc còn nhỏ, chi phí hằng số của phương pháp điểm-giá trị có thể lớn hơn. Khi bậc tăng, độ phức tạp $O(n \log n)$ trở nên vượt trội so với $O(n^2)$, dù cần chú ý sai số số thực trong tính toán với căn đơn vị phức.